SQL Phase : 2

🔗 Section 1 — JOINs & CASE

Q1. Customer Order Report

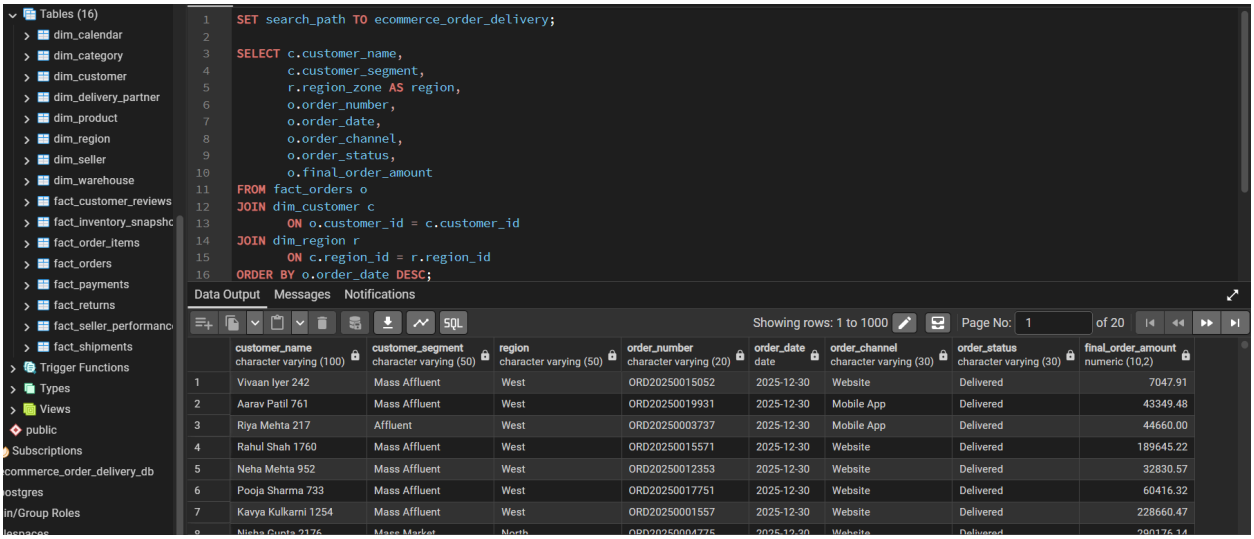
Using appropriate JOINs, display:

- Customer Name
- Customer Segment
- Region
- Order Number
- Order Date
- Order Channel
- Order Status
- Final Order Amount

Sort by **Order Date descending**.

```
SET search_path TO ecommerce_order_delivery;

SELECT c.customer_name,
       c.customer_segment,
       r.region_zone AS region,
       o.order_number,
       o.order_date,
       o.order_channel,
       o.order_status,
       o.final_order_amount
```

```sql
FROM fact_orders o
JOIN dim_customer c
       ON o.customer_id = c.customer_id
JOIN dim_region r
       ON c.region_id = r.region_id
ORDER BY o.order_date DESC;
```



Q2. Order Item Sales Report

Create a report showing:

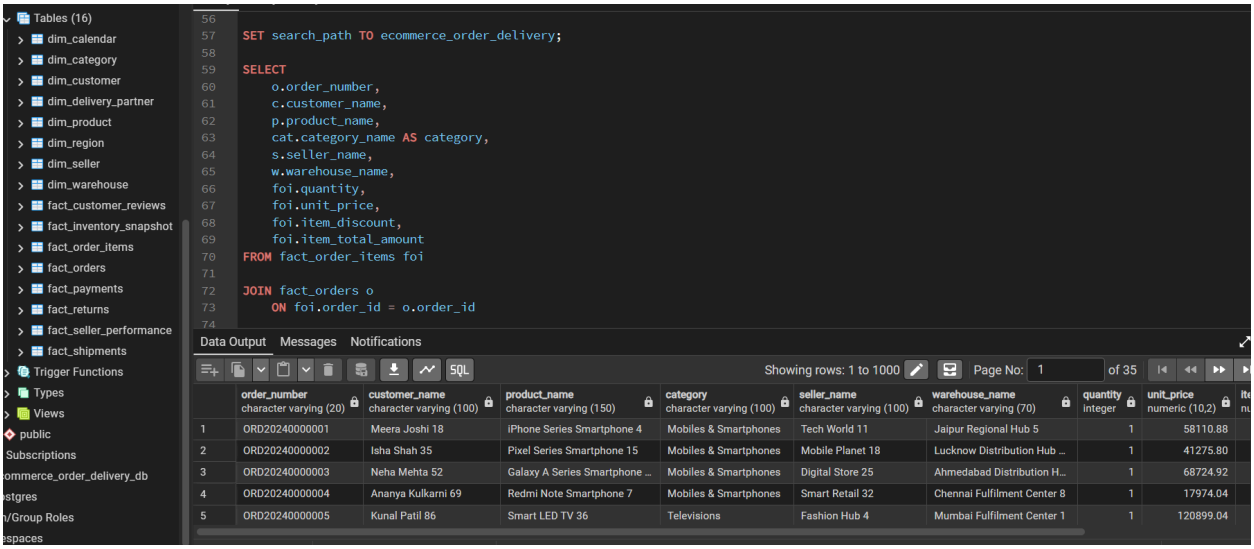
- Order Number
- Customer Name
- Product Name
- Category
- Seller Name
- Warehouse Name
- Quantity
- Unit Price

- Item Discount
- Item Total Amount

Use the required JOINs across the order, item, product, category, seller and warehouse tables.

```sql
SET search_path TO ecommerce_order_delivery;

SELECT
    o.order_number,
    c.customer_name,
    p.product_name,
    cat.category_name AS category,
    s.seller_name,
    w.warehouse_name,
    foi.quantity,
    foi.unit_price,
    foi.item_discount,
    foi.item_total_amount
FROM fact_order_items foi

JOIN fact_orders o
    ON foi.order_id = o.order_id

JOIN dim_customer c
    ON o.customer_id = c.customer_id

JOIN dim_product p
    ON foi.product_id = p.product_id

JOIN dim_category cat
    ON p.category_id = cat.category_id

JOIN dim_seller s
    ON foi.seller_id = s.seller_id
```

```sql
JOIN dim_warehouse w
    ON foi.warehouse_id = w.warehouse_id;
```



Q3. Product Sales Performance

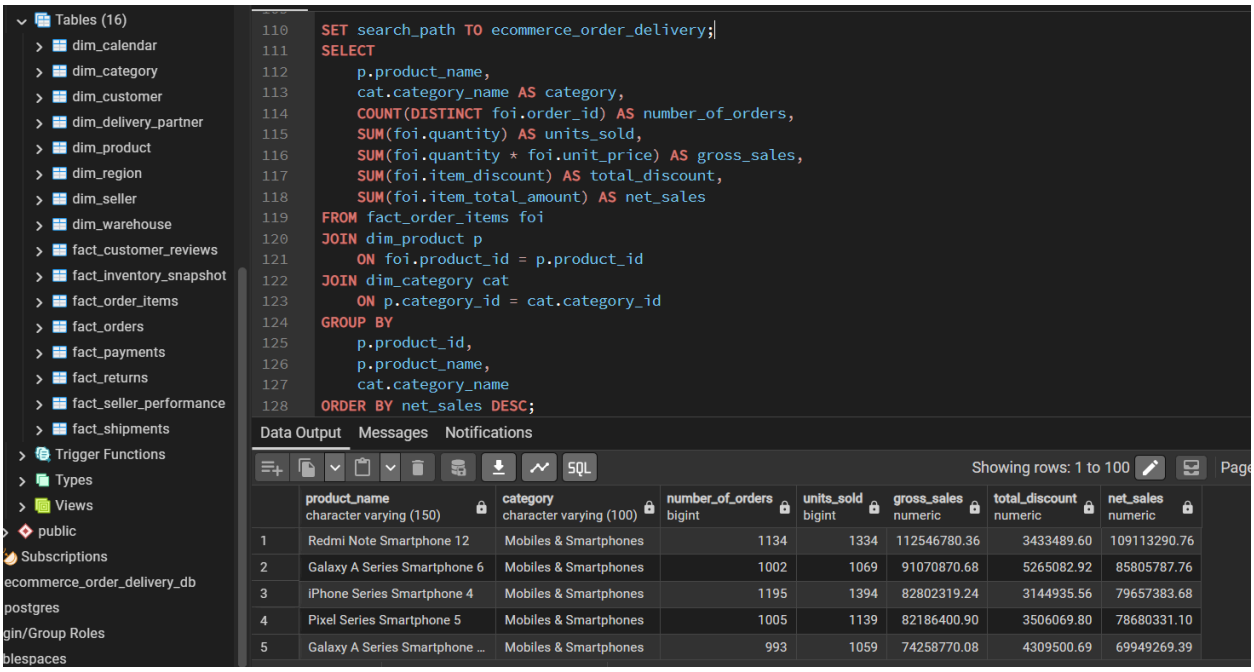
For every product calculate:

- Product Name
- Category
- Number of Orders
- Units Sold
- Gross Sales
- Total Discount
- Net Sales

Sort by **Net Sales descending**.

```sql
SET search_path TO ecommerce_order_delivery;

SELECT
```

```sql
    p.product_name,
    cat.category_name AS category,
    COUNT(DISTINCT foi.order_id) AS number_of_orders,
    SUM(foi.quantity) AS units_sold,
    SUM(foi.quantity * foi.unit_price) AS gross_sales,
    SUM(foi.item_discount) AS total_discount,
    SUM(foi.item_total_amount) AS net_sales
FROM fact_order_items foi
JOIN dim_product p
    ON foi.product_id = p.product_id
JOIN dim_category cat
    ON p.category_id = cat.category_id
GROUP BY
    p.product_id,
    p.product_name,
    cat.category_name
ORDER BY net_sales DESC;
```



Q4. Customer Spending Classification

Using `CASE`, classify customers based on their **total final order amount**:

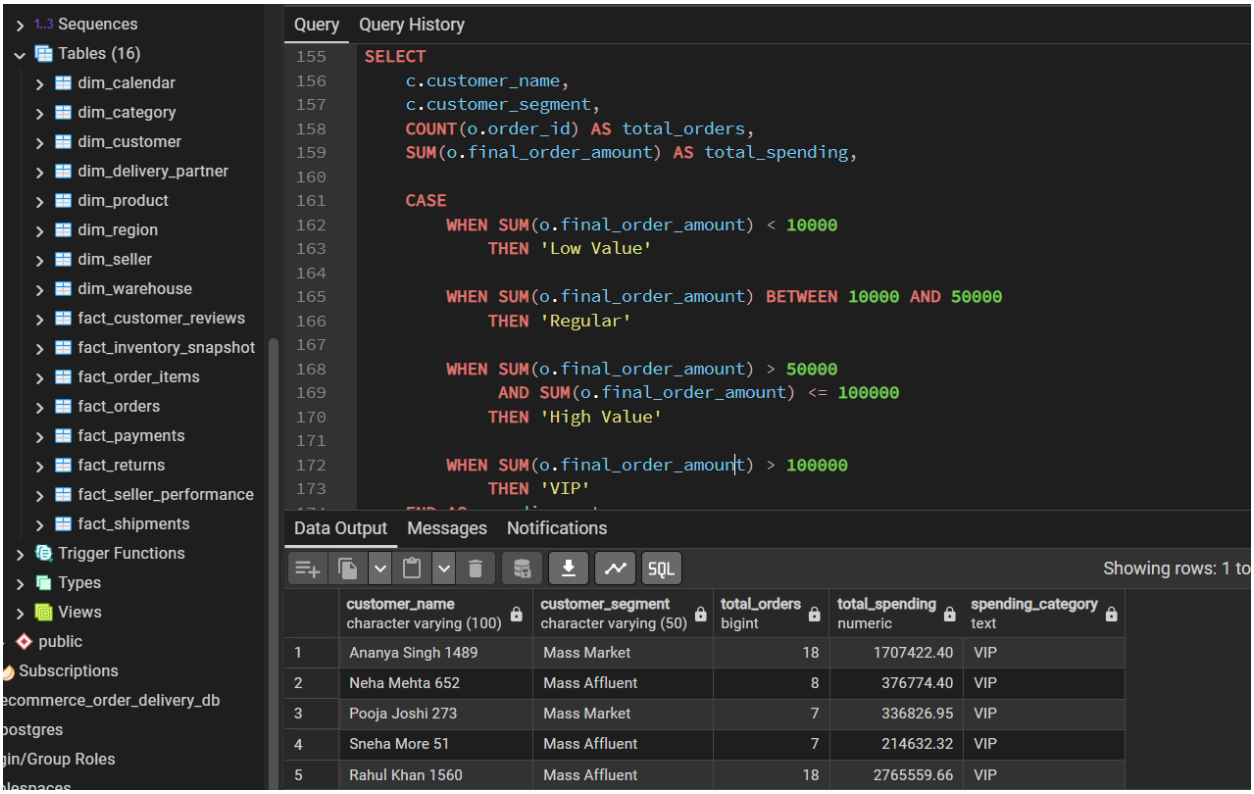
Total Spending	Customer Type
< ₹10,000	Low Value
₹10,000–₹50,000	Regular
₹50,000–₹1,00,000	High Value
> ₹1,00,000	VIP

Display:

- Customer Name
- Customer Segment
- Total Orders
- Total Spending
- Spending Category

```
SET search_path TO ecommerce_order_delivery;

SELECT
    c.customer_name,
    c.customer_segment,
    COUNT(o.order_id) AS total_orders,
    SUM(o.final_order_amount) AS total_spending,

    CASE
        WHEN SUM(o.final_order_amount) < 10000
            THEN 'Low Value'

        WHEN SUM(o.final_order_amount) BETWEEN 10000 AND 5000
0
            THEN 'Regular'
```

```sql
        WHEN SUM(o.final_order_amount) > 50000
             AND SUM(o.final_order_amount) <= 100000
            THEN 'High Value'

        WHEN SUM(o.final_order_amount) > 100000
            THEN 'VIP'
    END AS spending_category

FROM fact_orders o
JOIN dim_customer c
    ON o.customer_id = c.customer_id

GROUP BY
    c.customer_id,
    c.customer_name,
    c.customer_segment;
```



Q5. Order Value Classification

Using `CASE`, classify orders based on `final_order_amount`:

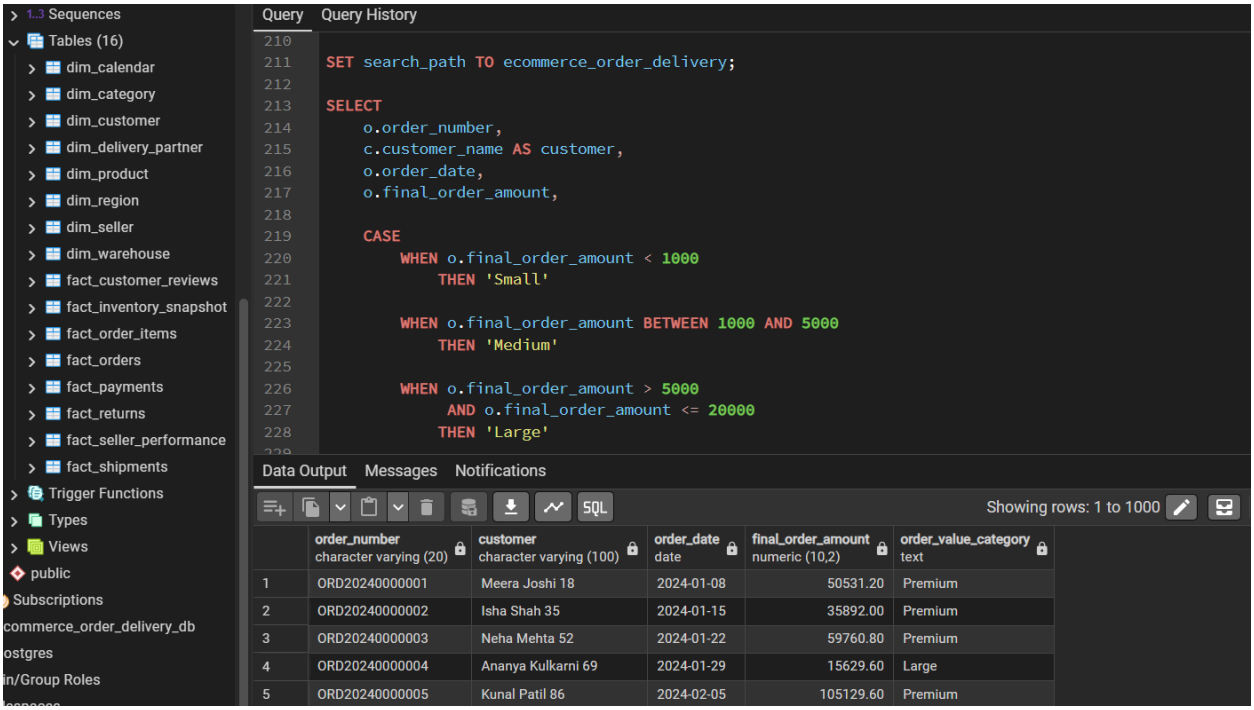
Order Amount	Category
< ₹1,000	Small
₹1,000–₹5,000	Medium
₹5,000–₹20,000	Large
> ₹20,000	Premium

Display:

- Order Number
- Customer
- Order Date
- Final Order Amount
- Order Value Category

```
SET search_path TO ecommerce_order_delivery;

SELECT
    o.order_number,
    c.customer_name AS customer,
    o.order_date,
    o.final_order_amount,

    CASE
        WHEN o.final_order_amount < 1000
            THEN 'Small'

        WHEN o.final_order_amount BETWEEN 1000 AND 5000
            THEN 'Medium'
```

```sql
        WHEN o.final_order_amount > 5000
             AND o.final_order_amount <= 20000
            THEN 'Large'

        WHEN o.final_order_amount > 20000
            THEN 'Premium'
    END AS order_value_category

FROM fact_orders o
JOIN dim_customer c
    ON o.customer_id = c.customer_id;
```



📅 Section 2 — Date Functions & Analysis

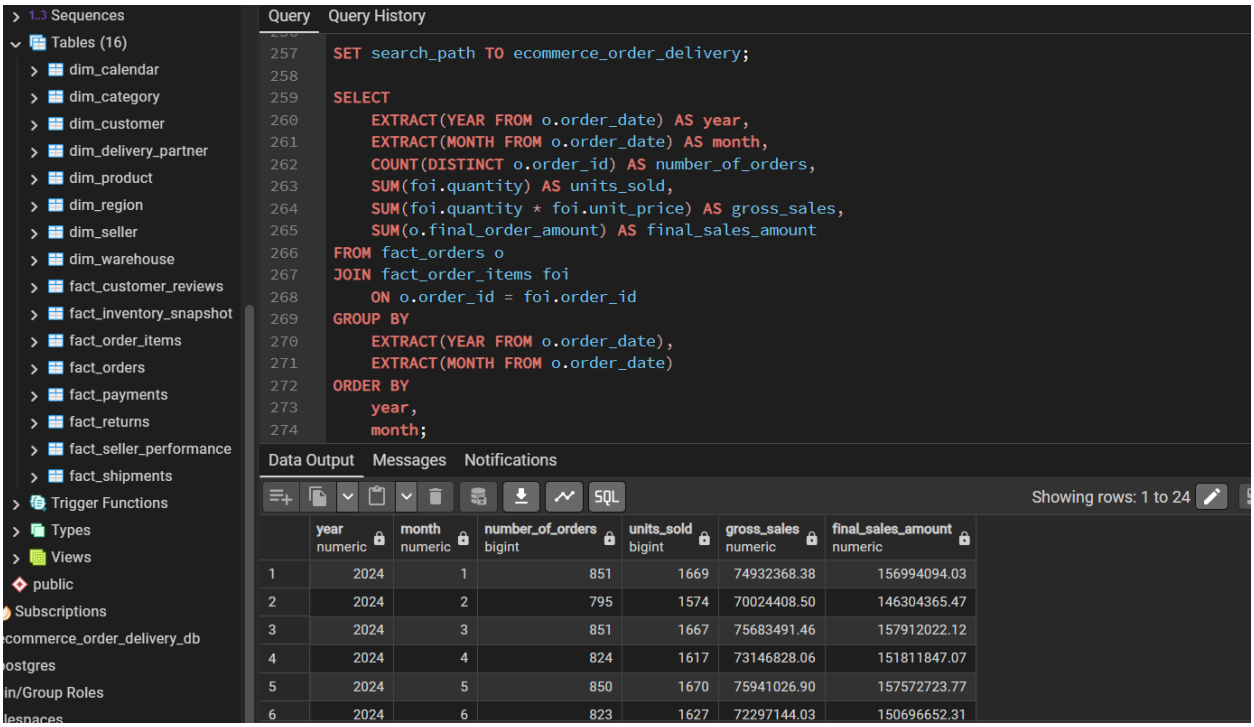
Q6. Monthly Sales Analysis

Using `TO_CHAR()`, `EXTRACT()` or `DATE_TRUNC()`, calculate monthly:

- Year
- Month
- Number of Orders
- Units Sold
- Gross Sales
- Final Sales Amount

Sort chronologically.

```
SET search_path TO ecommerce_order_delivery;

SELECT
    EXTRACT(YEAR FROM o.order_date) AS year,
    EXTRACT(MONTH FROM o.order_date) AS month,
    COUNT(DISTINCT o.order_id) AS number_of_orders,
    SUM(foi.quantity) AS units_sold,
    SUM(foi.quantity * foi.unit_price) AS gross_sales,
    SUM(o.final_order_amount) AS final_sales_amount
FROM fact_orders o
JOIN fact_order_items foi
    ON o.order_id = foi.order_id
GROUP BY
    EXTRACT(YEAR FROM o.order_date),
    EXTRACT(MONTH FROM o.order_date)
ORDER BY
    year,
    month;
```



Q7. Weekend vs Weekday Sales

Using the order date and `dim_calendar`, compare:

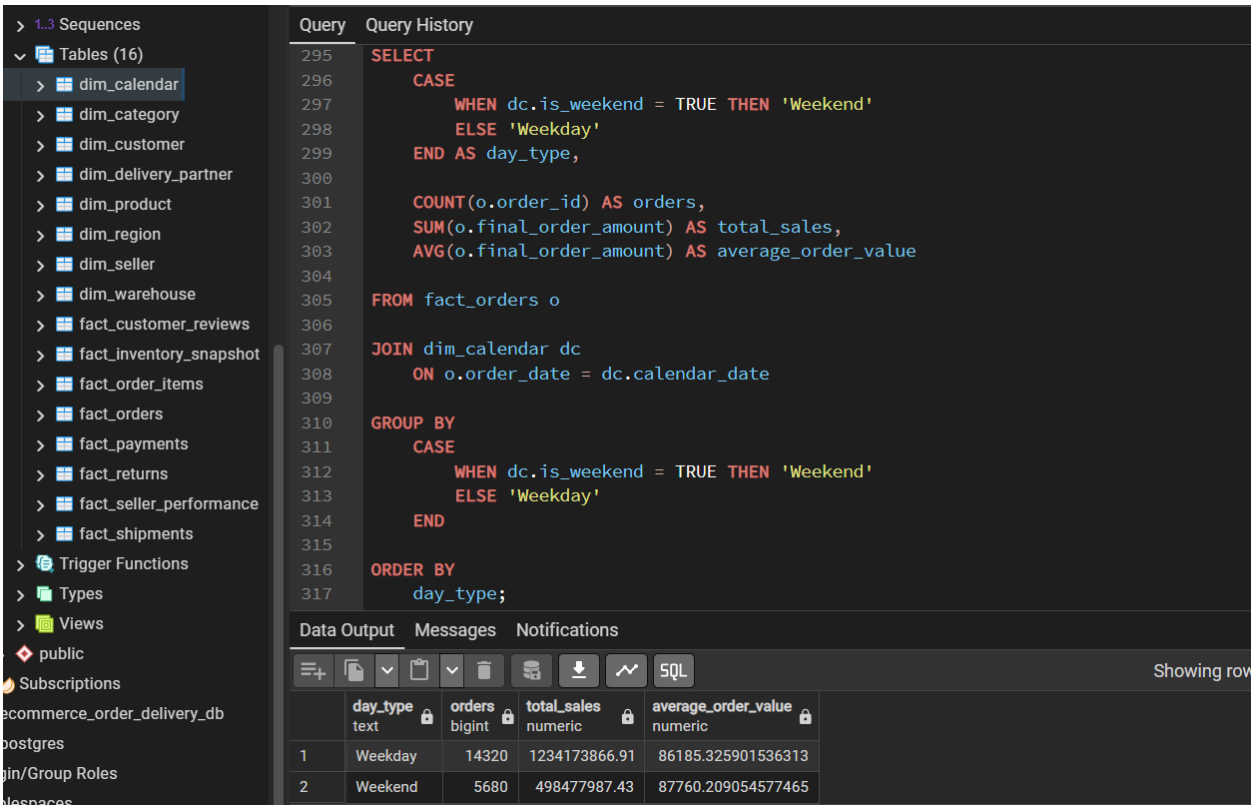
- Weekday Orders
- Weekend Orders
- Total Sales
- Average Order Value

Display the results as:

```
Day Type | Orders | Total Sales | Average Order Value
```

```sql
SELECT
    CASE
        WHEN dc.is_weekend = TRUE THEN 'Weekend'
        ELSE 'Weekday'
```

```sql
    END AS day_type,

    COUNT(o.order_id) AS orders,
    SUM(o.final_order_amount) AS total_sales,
    AVG(o.final_order_amount) AS average_order_value

FROM fact_orders o

JOIN dim_calendar dc
    ON o.order_date = dc.calendar_date

GROUP BY
    CASE
        WHEN dc.is_weekend = TRUE THEN 'Weekend'
        ELSE 'Weekday'
    END

ORDER BY
    day_type;
```



Q8. Delivery Delay Analysis

Using shipment dates, calculate the number of days between:

Estimated Delivery Date → Actual Delivery Date

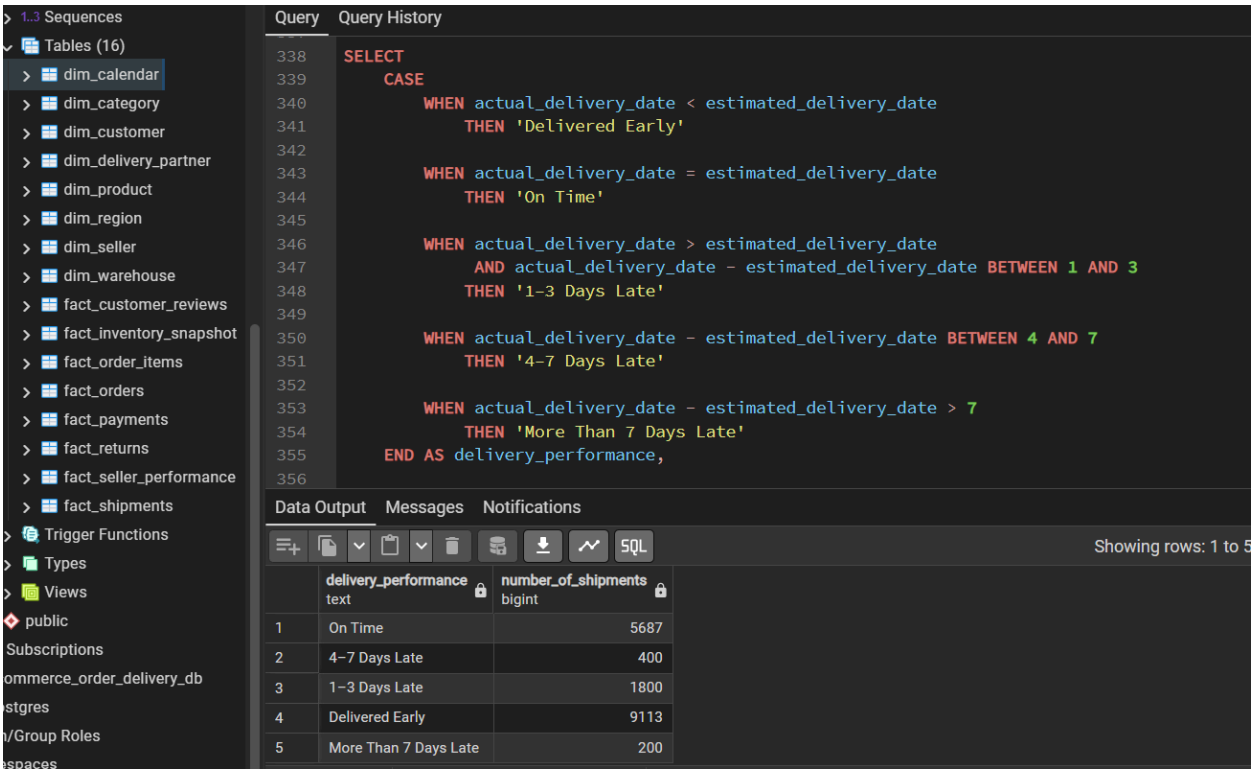
Classify delivery performance using `CASE`:

- Delivered Early
- On Time
- 1–3 Days Late
- 4–7 Days Late
- More Than 7 Days Late

Find the number of shipments in each category.

```sql
SELECT
    CASE
        WHEN actual_delivery_date < estimated_delivery_date
            THEN 'Delivered Early'

        WHEN actual_delivery_date = estimated_delivery_date
            THEN 'On Time'

        WHEN actual_delivery_date > estimated_delivery_date
             AND actual_delivery_date - estimated_delivery_da
te BETWEEN 1 AND 3
            THEN '1–3 Days Late'

        WHEN actual_delivery_date - estimated_delivery_date B
ETWEEN 4 AND 7
            THEN '4–7 Days Late'

        WHEN actual_delivery_date - estimated_delivery_date >
7
            THEN 'More Than 7 Days Late'
    END AS delivery_performance,

    COUNT(shipment_id) AS number_of_shipments

FROM fact_shipments

WHERE actual_delivery_date IS NOT NULL

GROUP BY
    CASE
        WHEN actual_delivery_date < estimated_delivery_date
            THEN 'Delivered Early'

        WHEN actual_delivery_date = estimated_delivery_date
            THEN 'On Time'
```

```
        WHEN actual_delivery_date > estimated_delivery_date
             AND actual_delivery_date - estimated_delivery_da
te BETWEEN 1 AND 3
            THEN '1–3 Days Late'

        WHEN actual_delivery_date - estimated_delivery_date B
ETWEEN 4 AND 7
            THEN '4–7 Days Late'

        WHEN actual_delivery_date - estimated_delivery_date >
7
            THEN 'More Than 7 Days Late'
    END;
```



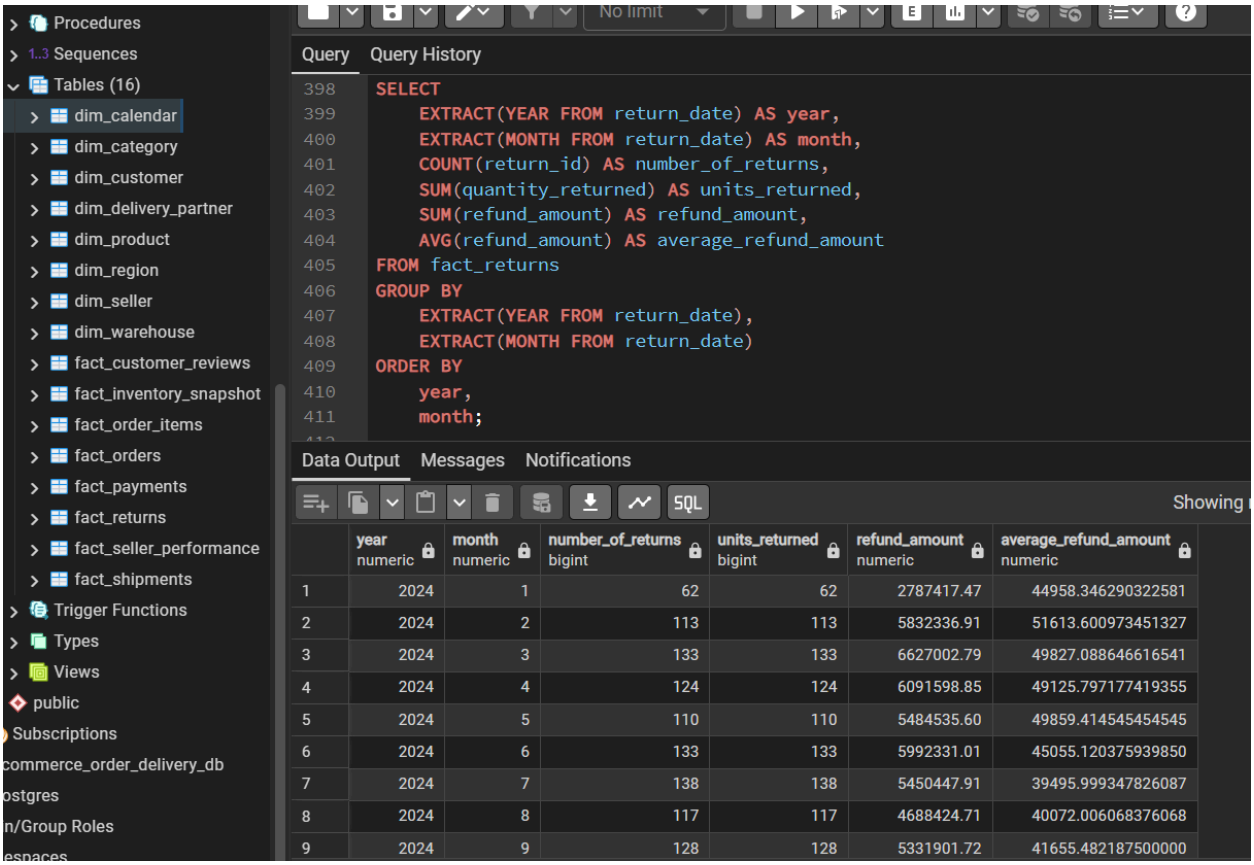
Q9. Return Analysis by Month

Using `return_date`, calculate monthly:

- Number of Returns
- Units Returned
- Refund Amount
- Average Refund Amount

Also identify the month with the **highest refund amount**.

```sql
SELECT
    EXTRACT(YEAR FROM return_date) AS year,
    EXTRACT(MONTH FROM return_date) AS month,
    COUNT(return_id) AS number_of_returns,
    SUM(quantity_returned) AS units_returned,
    SUM(refund_amount) AS refund_amount,
    AVG(refund_amount) AS average_refund_amount
FROM fact_returns
GROUP BY
    EXTRACT(YEAR FROM return_date),
    EXTRACT(MONTH FROM return_date)
ORDER BY
    year,
    month;
```



🧩 Section 3 — CTE

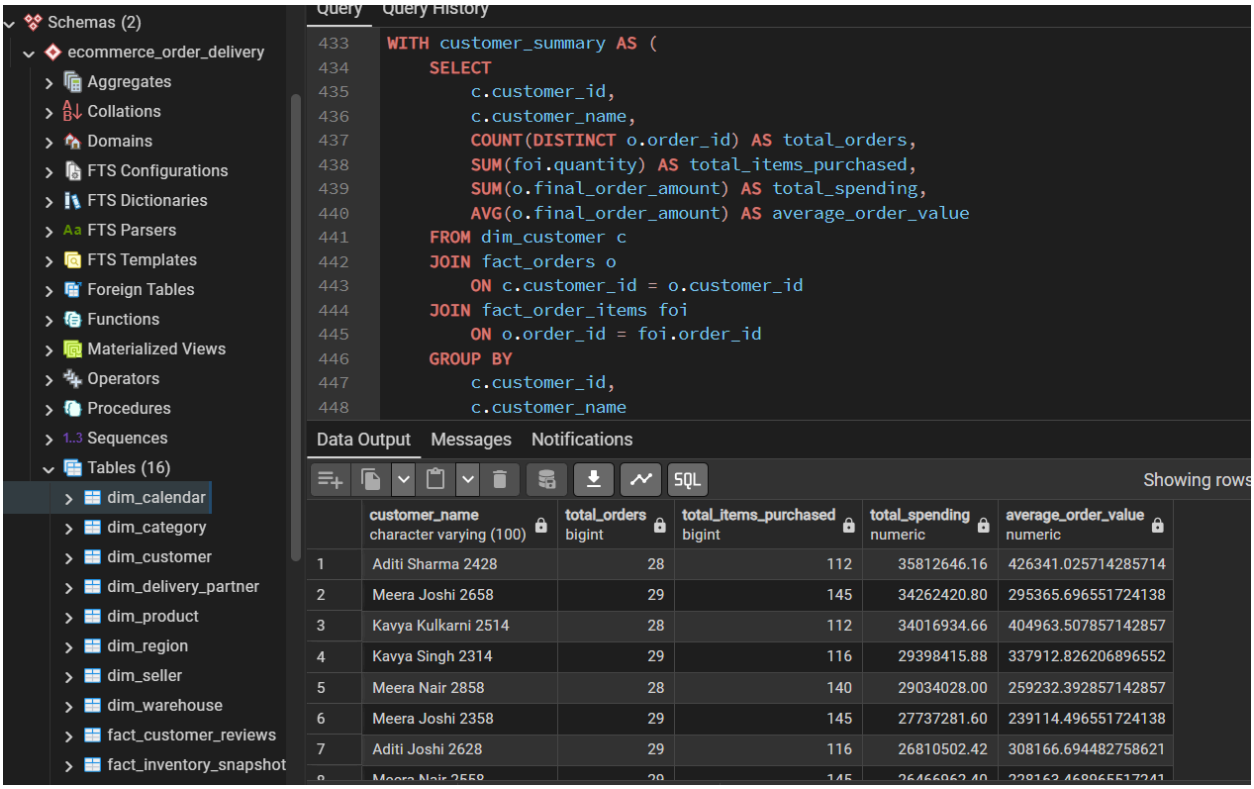
Q10. Customer Sales Summary Using CTE

Create a CTE that calculates for every customer:

- Total Orders
- Total Items Purchased
- Total Spending
- Average Order Value

Return customers whose spending is **above the overall average customer spending**.

```sql
SET search_path TO ecommerce_order_delivery;

WITH customer_summary AS (
    SELECT
        c.customer_id,
        c.customer_name,
        COUNT(DISTINCT o.order_id) AS total_orders,
        SUM(foi.quantity) AS total_items_purchased,
        SUM(o.final_order_amount) AS total_spending,
        AVG(o.final_order_amount) AS average_order_value
    FROM dim_customer c
    JOIN fact_orders o
        ON c.customer_id = o.customer_id
    JOIN fact_order_items foi
        ON o.order_id = foi.order_id
    GROUP BY
        c.customer_id,
        c.customer_name
)

SELECT
    customer_name,
    total_orders,
    total_items_purchased,
    total_spending,
    average_order_value
FROM customer_summary
WHERE total_spending > (
    SELECT AVG(total_spending)
    FROM customer_summary
)
ORDER BY total_spending DESC;
```



Q11. Category Performance Using CTE

Create a CTE that calculates for every category:

- Number of Orders
- Units Sold
- Gross Sales
- Net Sales
- Return Quantity

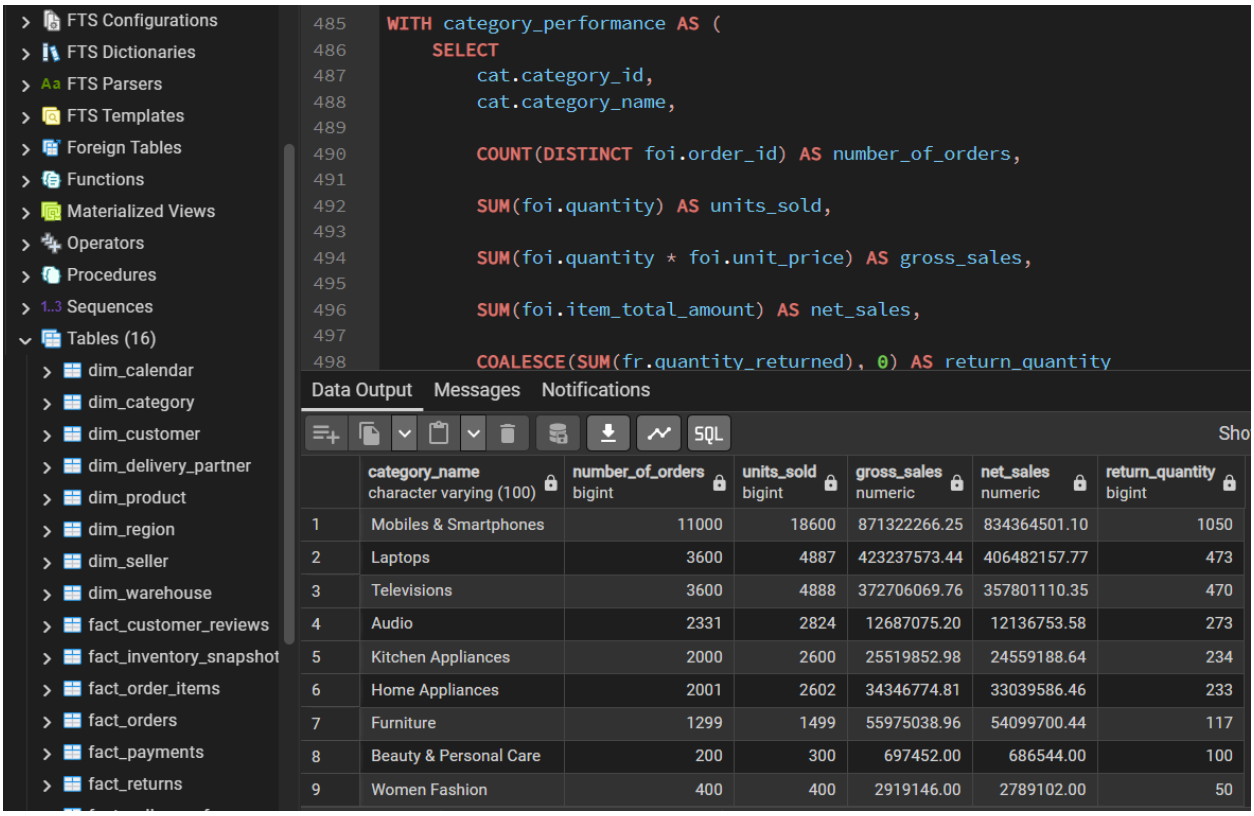
Identify categories where:

Return Quantity > 5% of Units Sold

```
SET search_path TO ecommerce_order_delivery;
```

```sql
WITH category_performance AS (
    SELECT
        cat.category_id,
        cat.category_name,

        COUNT(DISTINCT foi.order_id) AS number_of_orders,

        SUM(foi.quantity) AS units_sold,

        SUM(foi.quantity * foi.unit_price) AS gross_sales,

        SUM(foi.item_total_amount) AS net_sales,

        COALESCE(SUM(fr.quantity_returned), 0) AS return_quan
tity

    FROM dim_category cat

    JOIN dim_product p
        ON cat.category_id = p.category_id

    JOIN fact_order_items foi
        ON p.product_id = foi.product_id

    LEFT JOIN fact_returns fr
        ON foi.order_item_id = fr.order_item_id

    GROUP BY
        cat.category_id,
        cat.category_name
)

SELECT
    category_name,
    number_of_orders,
    units_sold,
```

```
    gross_sales,
    net_sales,
    return_quantity
FROM category_performance
WHERE return_quantity > units_sold * 0.05
ORDER BY return_quantity DESC;
```



	category_name character varying (100)	number_of_orders bigint	units_sold bigint	gross_sales numeric	net_sales numeric	return_quantity bigint
1	Mobiles & Smartphones	11000	18600	871322266.25	834364501.10	1050
2	Laptops	3600	4887	423237573.44	406482157.77	473
3	Televisions	3600	4888	372706069.76	357801110.35	470
4	Audio	2331	2824	12687075.20	12136753.58	273
5	Kitchen Appliances	2000	2600	25519852.98	24559188.64	234
6	Home Appliances	2001	2602	34346774.81	33039586.46	233
7	Furniture	1299	1499	55975038.96	54099700.44	117
8	Beauty & Personal Care	200	300	697452.00	686544.00	100
9	Women Fashion	400	400	2919146.00	2789102.00	50

Q12. Seller Performance Using CTE

Using `fact_seller_performance`, create a CTE to calculate for every seller:

- Total Orders
- Units Sold
- Gross Sales
- Returns

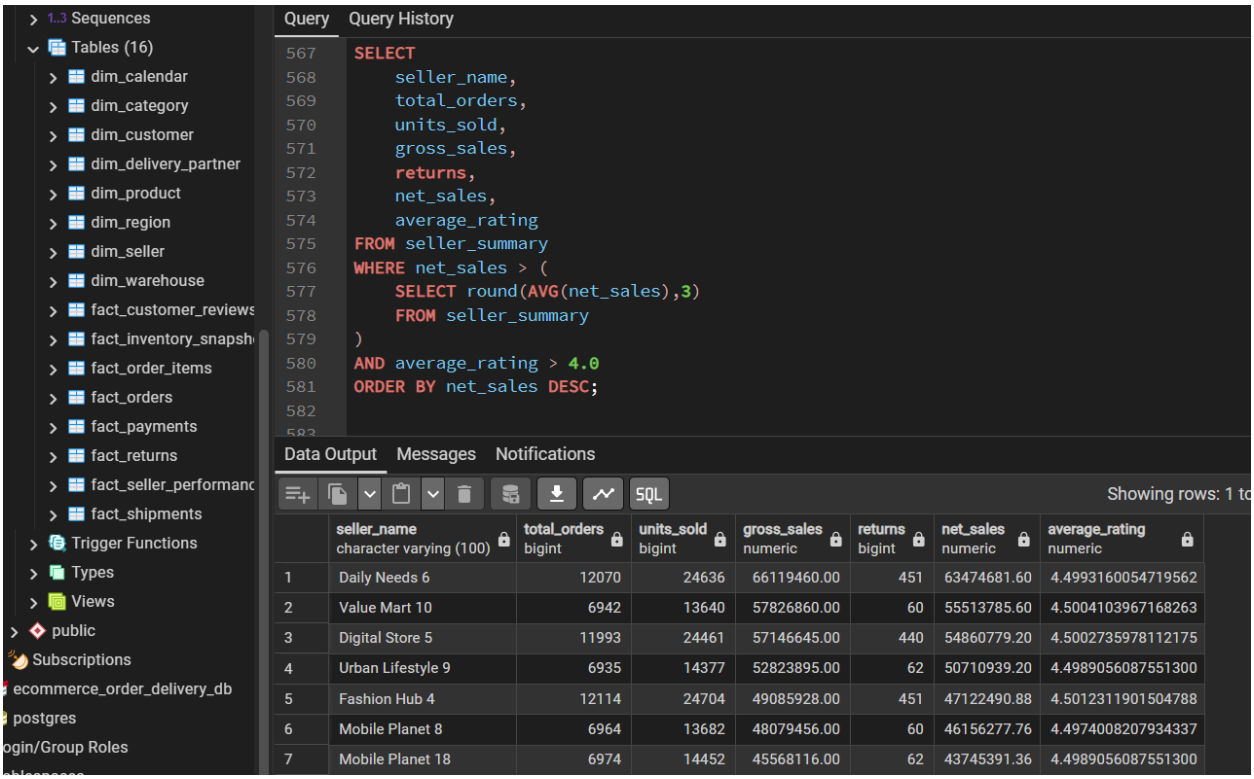
- Net Sales
- Average Rating

Then identify sellers with:

- Net Sales above average
- Rating above 4.0

```sql
SET search_path TO ecommerce_order_delivery;

WITH seller_summary AS (
    SELECT
        s.seller_id,
        s.seller_name,
        SUM(fsp.orders_count) AS total_orders,
        SUM(fsp.units_sold) AS units_sold,
        SUM(fsp.gross_sales_amount) AS gross_sales,
        SUM(fsp.returns_count) AS returns,
        SUM(fsp.net_sales_amount) AS net_sales,
        AVG(fsp.rating_average) AS average_rating
    FROM fact_seller_performance fsp
    JOIN dim_seller s
        ON fsp.seller_id = s.seller_id
    GROUP BY
        s.seller_id,
        s.seller_name
)

SELECT
    seller_name,
    total_orders,
    units_sold,
    gross_sales,
    returns,
    net_sales,
```

```sql
    average_rating
FROM seller_summary
WHERE net_sales > (
    SELECT AVG(net_sales)
    FROM seller_summary
)
AND average_rating > 4.0
ORDER BY net_sales DESC;
```



Q13. Delivery Partner Performance

Using a CTE, calculate for every delivery partner:

- Total Shipments
- Delivered Shipments

- Delayed Shipments
- Average Delivery Delay
- Shipping Cost

Calculate:

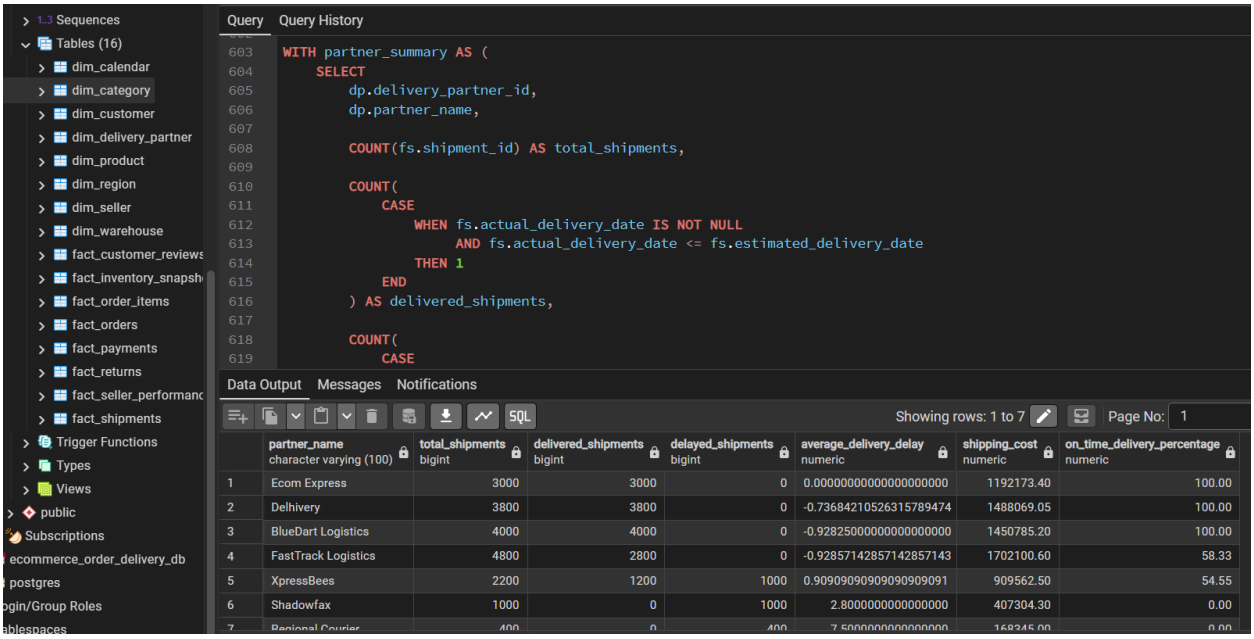
On-Time Delivery %

Identify the best-performing delivery partner.

```
WITH partner_summary AS (
    SELECT
        dp.delivery_partner_id,
        dp.partner_name,

        COUNT(fs.shipment_id) AS total_shipments,

        COUNT(
            CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                     AND fs.actual_delivery_date <= fs.estima
ted_delivery_date
                THEN 1
            END
        ) AS delivered_shipments,

        COUNT(
            CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                     AND fs.actual_delivery_date > fs.estimat
ed_delivery_date
                THEN 1
            END
        ) AS delayed_shipments,
```

```
        AVG(
            CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                THEN fs.actual_delivery_date - fs.estimated_d
elivery_date
            END
        ) AS average_delivery_delay,

        SUM(fs.shipping_cost) AS shipping_cost

    FROM dim_delivery_partner dp

    JOIN fact_shipments fs
        ON dp.delivery_partner_id = fs.delivery_partner_id

    GROUP BY
        dp.delivery_partner_id,
        dp.partner_name
)

SELECT
    partner_name,
    total_shipments,
    delivered_shipments,
    delayed_shipments,
    average_delivery_delay,
    shipping_cost,

    ROUND(
        delivered_shipments * 100.0 / NULLIF(total_shipments,
0),
        2
    ) AS on_time_delivery_percentage

FROM partner_summary
```

```sql
ORDER BY
    on_time_delivery_percentage DESC;
```



Q14. Product Return Analysis

Using a CTE:

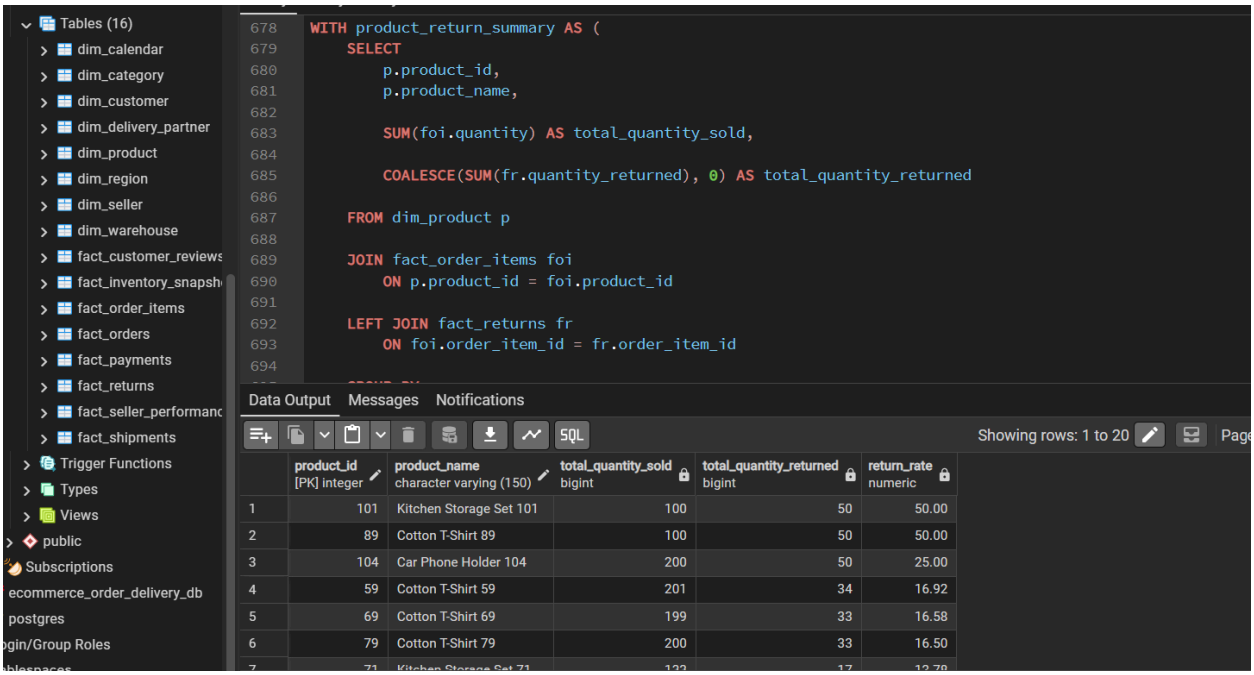
1. Calculate total quantity sold for every product.
2. Calculate total quantity returned.
3. Calculate return rate.

Return the **Top 20 products with the highest return rate**, considering only products with at least **20 units sold**.

```sql
WITH product_return_summary AS (
    SELECT
        p.product_id,
```

```sql
        p.product_name,

        SUM(foi.quantity) AS total_quantity_sold,

        COALESCE(SUM(fr.quantity_returned), 0) AS total_quant
ity_returned

    FROM dim_product p

    JOIN fact_order_items foi
        ON p.product_id = foi.product_id

    LEFT JOIN fact_returns fr
        ON foi.order_item_id = fr.order_item_id

    GROUP BY
        p.product_id,
        p.product_name
)

SELECT
    product_id,
    product_name,
    total_quantity_sold,
    total_quantity_returned,

    ROUND(
        total_quantity_returned * 100.0
        / NULLIF(total_quantity_sold, 0),
        2
    ) AS return_rate

FROM product_return_summary

WHERE total_quantity_sold >= 20
```

```
ORDER BY return_rate DESC

LIMIT 20;
```



👁️ Section 4 — Views

Q15. Customer Sales Summary View

Create a view:

`vw_customer_sales_summary`

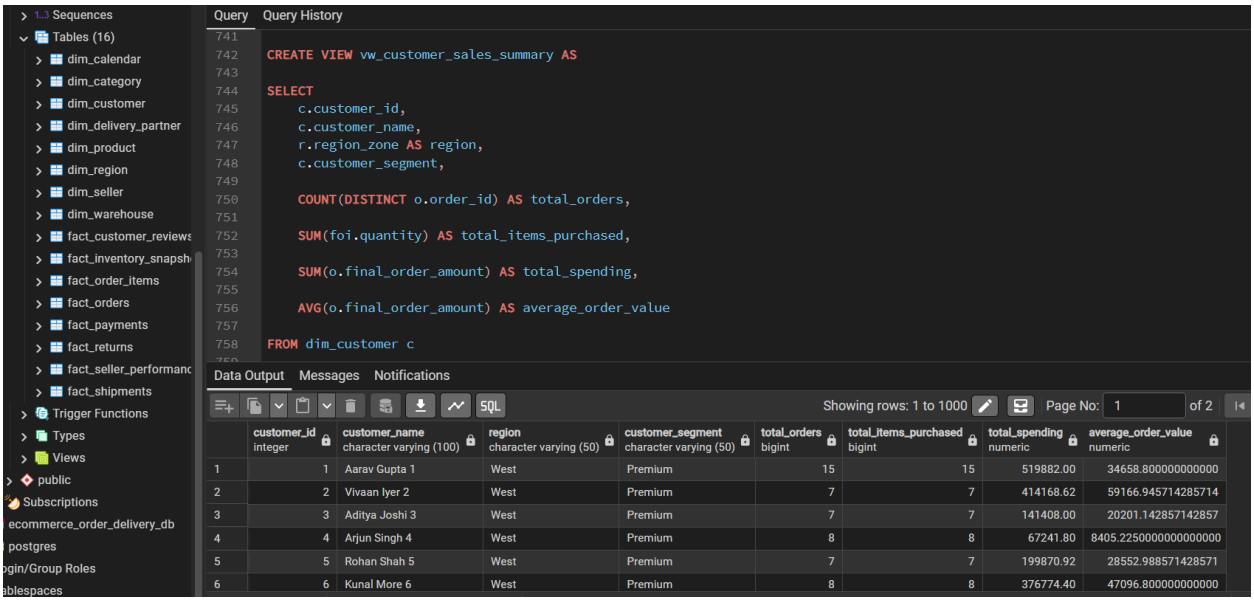
It should contain:

- Customer ID
- Customer Name
- Region

- Customer Segment
- Total Orders
- Total Items Purchased
- Total Spending
- Average Order Value

```sql
CREATE VIEW vw_customer_sales_summary AS

SELECT
    c.customer_id,
    c.customer_name,
    r.region_zone AS region,
    c.customer_segment,

    COUNT(DISTINCT o.order_id) AS total_orders,

    SUM(foi.quantity) AS total_items_purchased,

    SUM(o.final_order_amount) AS total_spending,

    AVG(o.final_order_amount) AS average_order_value

FROM dim_customer c

JOIN dim_region r
    ON c.region_id = r.region_id

JOIN fact_orders o
    ON c.customer_id = o.customer_id

JOIN fact_order_items foi
    ON o.order_id = foi.order_id
```

```sql
GROUP BY
    c.customer_id,
    c.customer_name,
    r.region_zone,
    c.customer_segment;

SELECT *
FROM vw_customer_sales_summary;
```



Q16. Product Performance View

Create:

`vw_product_sales_performance`

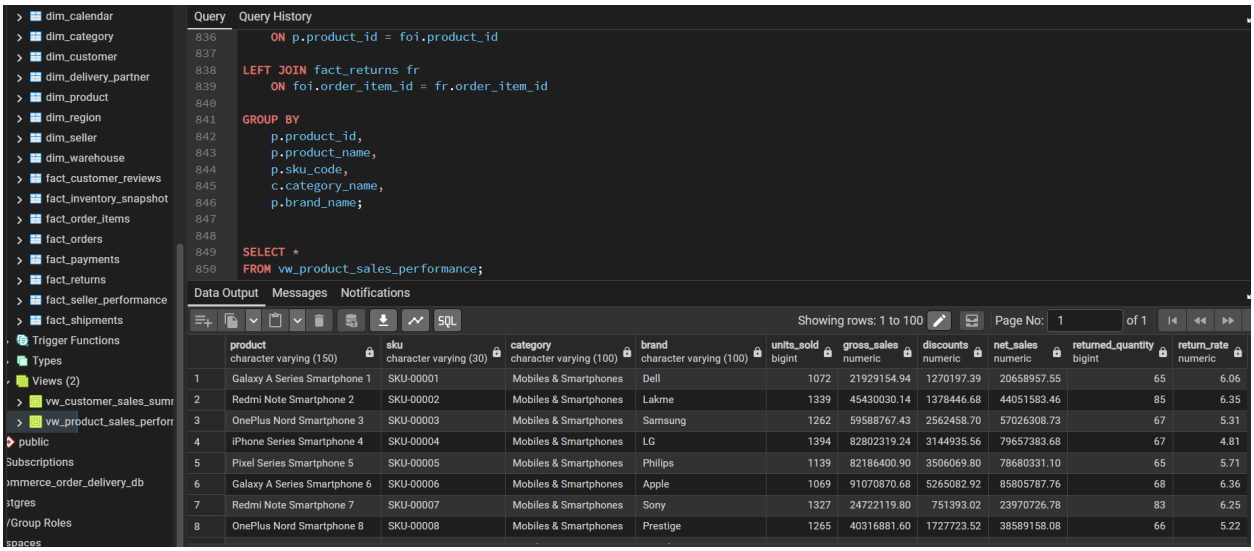
Include:

- Product
- SKU
- Category

- Brand
- Units Sold
- Gross Sales
- Discounts
- Net Sales
- Returned Quantity
- Return Rate

```
CREATE VIEW vw_product_sales_performance AS

SELECT
    p.product_name AS product,
    p.sku_code AS sku,
    c.category_name AS category,
    p.brand_name AS brand,

    SUM(foi.quantity) AS units_sold,

    SUM(foi.quantity * foi.unit_price) AS gross_sales,

    SUM(foi.item_discount) AS discounts,

    SUM(foi.item_total_amount) AS net_sales,

    COALESCE(SUM(fr.quantity_returned), 0) AS returned_quanti
ty,

    ROUND(
        COALESCE(SUM(fr.quantity_returned), 0) * 100.0
        / NULLIF(SUM(foi.quantity), 0),
        2
    ) AS return_rate
```

```sql
FROM dim_product p

JOIN dim_category c
    ON p.category_id = c.category_id

JOIN fact_order_items foi
    ON p.product_id = foi.product_id

LEFT JOIN fact_returns fr
    ON foi.order_item_id = fr.order_item_id

GROUP BY
    p.product_id,
    p.product_name,
    p.sku_code,
    c.category_name,
    p.brand_name;


SELECT *
FROM vw_product_sales_performance;
```



Q17. Delivery Performance View

Create:

`vw_delivery_performance`

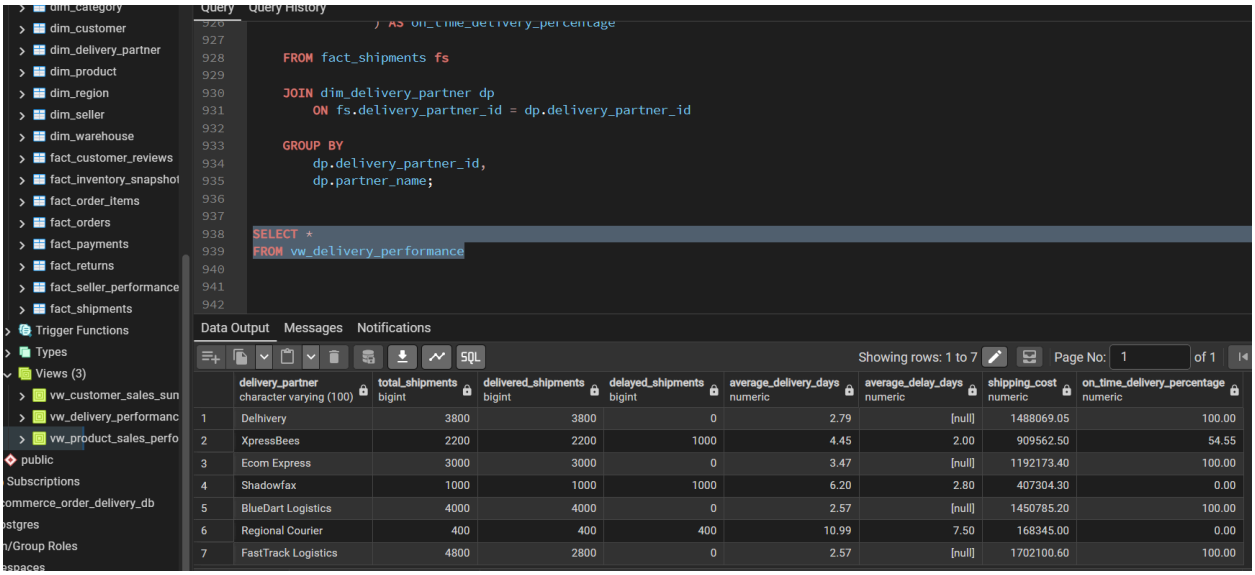
Include:

- Delivery Partner
- Total Shipments
- Delivered Shipments
- Delayed Shipments
- Average Delivery Days
- Average Delay Days
- Shipping Cost
- On-Time Delivery %

```
CREATE VIEW vw_delivery_performance AS

SELECT
```

```
    dp.partner_name AS delivery_partner,

    COUNT(fs.shipment_id) AS total_shipments,

    COUNT(CASE
            WHEN fs.actual_delivery_date IS NOT NULL
            THEN 1
        END )
        AS delivered_shipments,

    COUNT(CASE
            WHEN fs.actual_delivery_date IS NOT NULL
                 AND fs.actual_delivery_date > fs.estimated_d
elivery_date
            THEN 1
        END )
        AS delayed_shipments,

    ROUND(AVG(CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                THEN fs.actual_delivery_date - fs.shipment_da
te
            END ),2)
            AS average_delivery_days,

    ROUND(
        AVG(CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                     AND fs.actual_delivery_date > fs.estimat
ed_delivery_date
                THEN fs.actual_delivery_date
                     - fs.estimated_delivery_date
            END),2)
                AS average_delay_days,

        SUM(fs.shipping_cost) AS shipping_cost,
```

```
        ROUND(COUNT(CASE
                        WHEN fs.actual_delivery_date IS NOT N
ULL
                             AND fs.actual_delivery_date <= f
s.estimated_delivery_date
                        THEN 1
                    END ) * 100.0 / NULLIF(
                        COUNT(
                            CASE
                                WHEN fs.actual_delivery_date
IS NOT NULL
                                THEN 1
                            END ),0),2) AS on_time_delivery_p
ercentage

    FROM fact_shipments fs

    JOIN dim_delivery_partner dp
        ON fs.delivery_partner_id = dp.delivery_partner_id

    GROUP BY
        dp.delivery_partner_id,
        dp.partner_name;


SELECT *
FROM vw_delivery_performance
```



Q18. View-Based Customer Analysis

Using `vw_customer_sales_summary`, find the **Top 20 customers by total spending**.

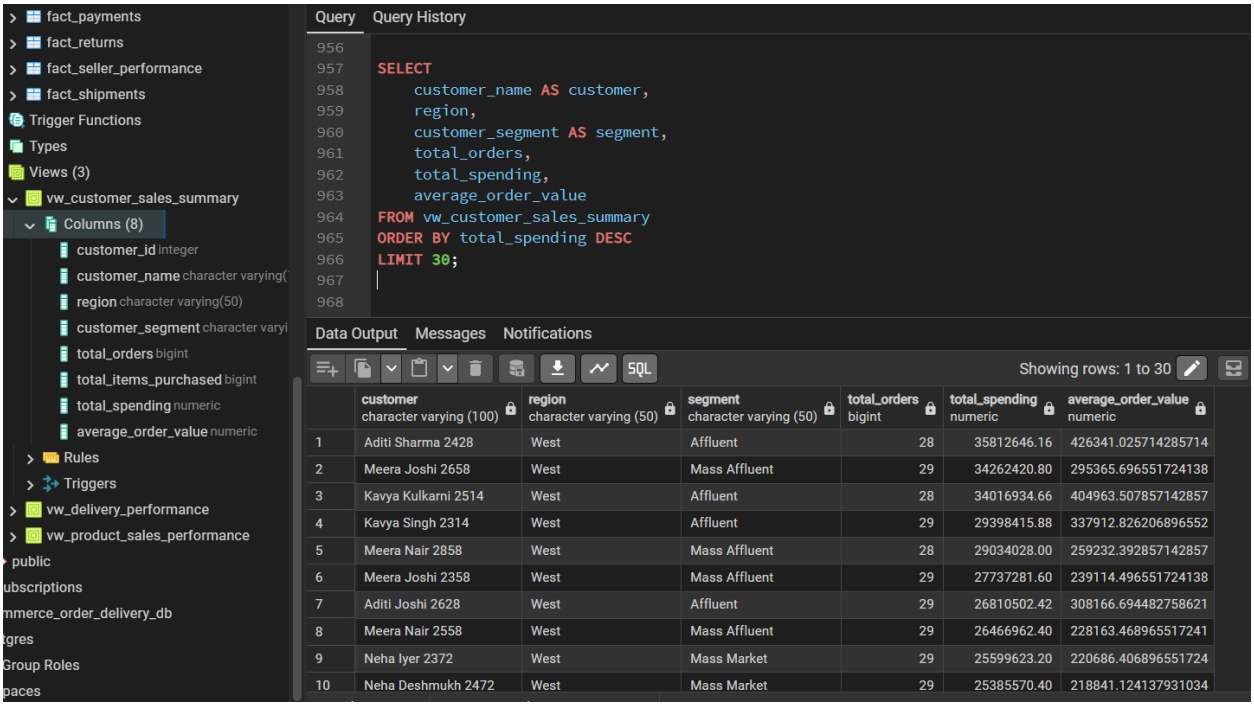
Display:

- Customer
- Region
- Segment
- Total Orders
- Total Spending
- Average Order Value

> Do not directly query the underlying customer/order tables for this question.

```
SELECT
customer_name AS customer,
region,
customer_segment AS segment,
```

```
    total_orders,
    total_spending,
    average_order_value
FROM vw_customer_sales_summary
ORDER BY total_spending DESC
LIMIT 30;
```



⚙️ Section 5 — PostgreSQL Functions

Q19. Customer Spending Function

Create:

`fn_customer_total_spending(customer_id)`

The function should return the customer's:

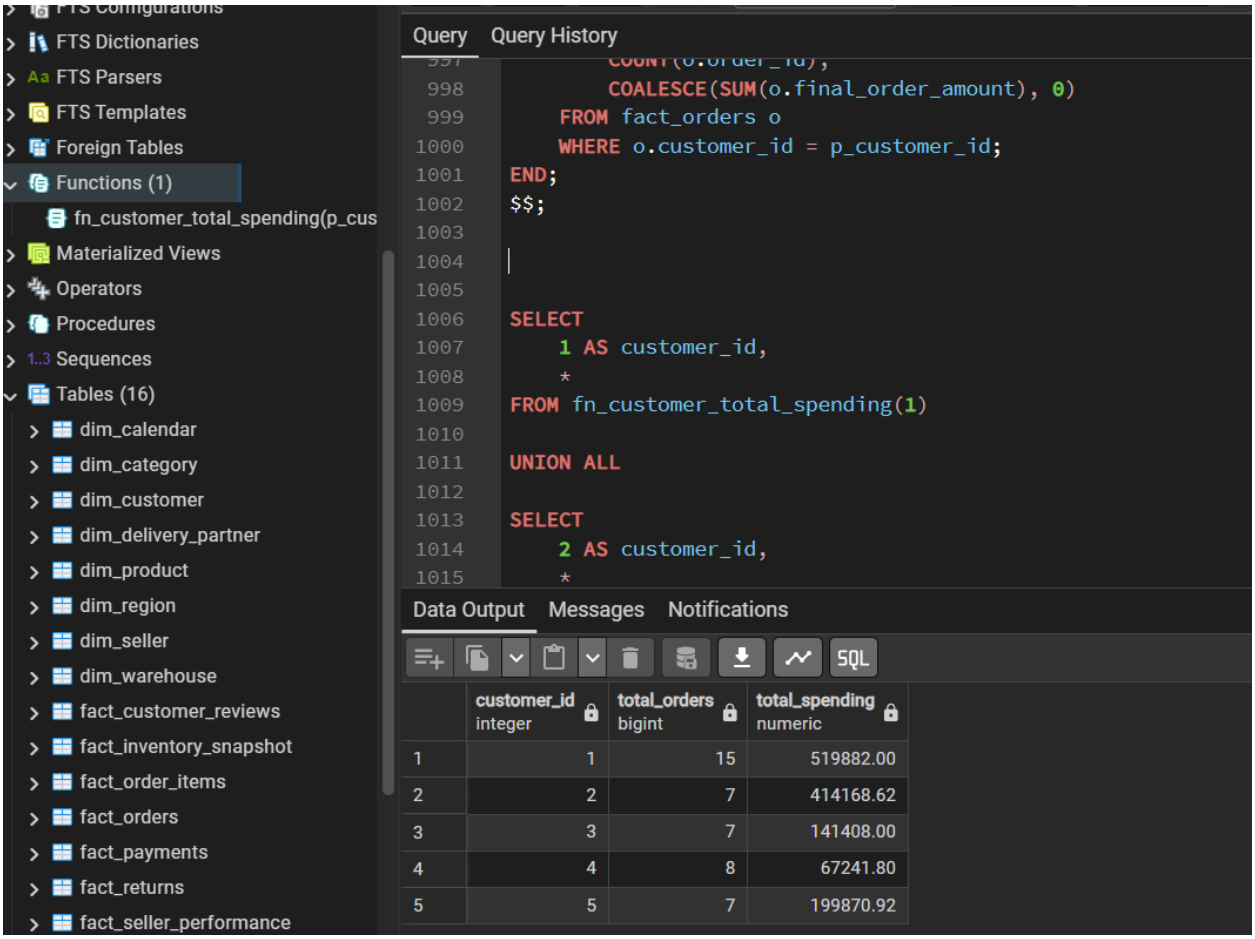
- Total Orders

- Total Spending

Test the function for at least **5 customers**.

```
CREATE OR REPLACE FUNCTION fn_customer_total_spending(p_custo
mer_id INT)
RETURNS TABLE (
    total_orders BIGINT,
    total_spending DECIMAL(10,2)
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        COUNT(o.order_id),
        COALESCE(SUM(o.final_order_amount), 0)
    FROM fact_orders o
    WHERE o.customer_id = p_customer_id;
END;
$$;



SELECT
    1 AS customer_id,
    *
FROM fn_customer_total_spending(1)

UNION ALL

SELECT
    2 AS customer_id,
    *
FROM fn_customer_total_spending(2)
```

```sql
UNION ALL

SELECT
    3 AS customer_id,
    *
FROM fn_customer_total_spending(3)

UNION ALL

SELECT
    4 AS customer_id,
    *
FROM fn_customer_total_spending(4)

UNION ALL

SELECT
    5 AS customer_id,
    *
FROM fn_customer_total_spending(5);
```



Q20. Order Value Classification Function

Create:

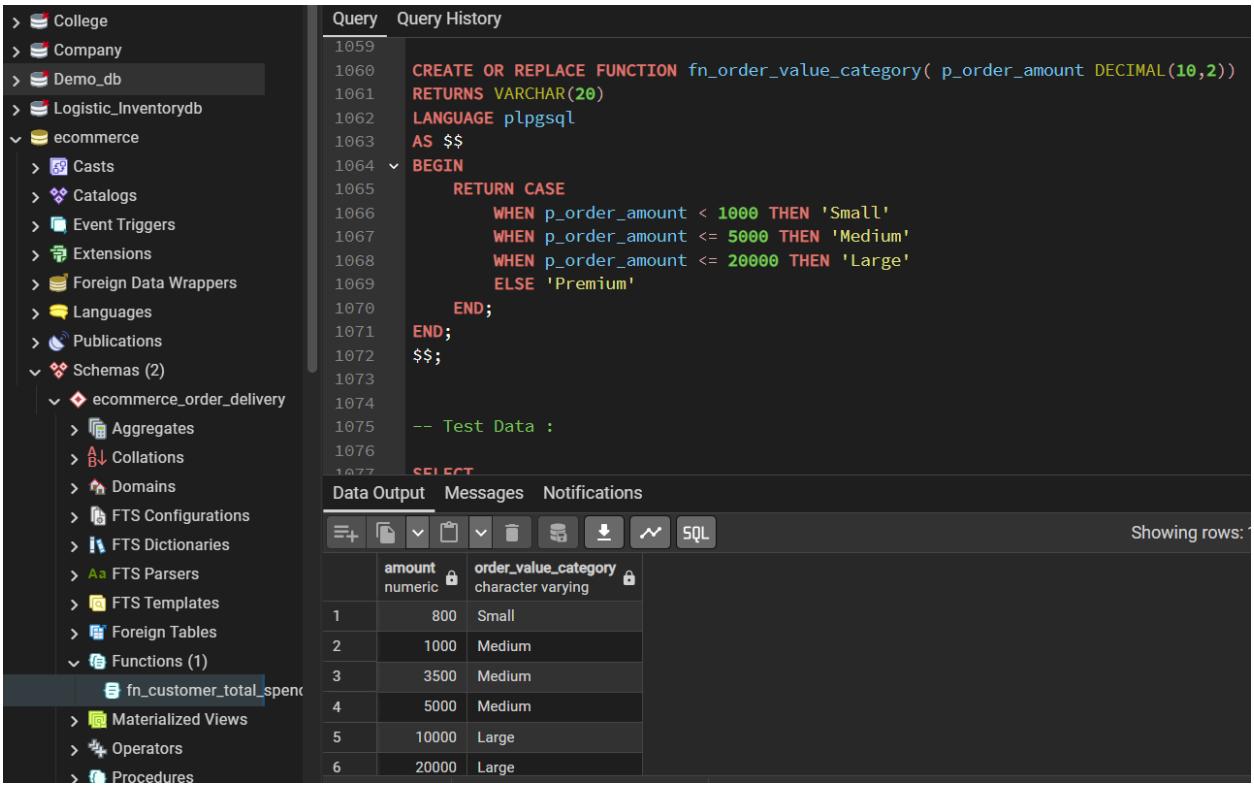
`fn_order_value_category(order_amount)`

The function should return:

Amount	Category
< ₹1,000	Small
₹1,000–₹5,000	Medium
₹5,000–₹20,000	Large
> ₹20,000	Premium

Test the function with multiple order amounts.

```sql
CREATE OR REPLACE FUNCTION fn_order_value_category( p_order_a
mount DECIMAL(10,2))
RETURNS VARCHAR(20)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN CASE
        WHEN p_order_amount < 1000 THEN 'Small'
        WHEN p_order_amount <= 5000 THEN 'Medium'
        WHEN p_order_amount <= 20000 THEN 'Large'
        ELSE 'Premium'
    END;
END;
$$;


-- Test Data :

SELECT
    amount,
    fn_order_value_category(amount) AS order_value_category
FROM (
    VALUES
        (800::DECIMAL),
        (1000::DECIMAL),
        (3500::DECIMAL),
        (5000::DECIMAL),
        (10000::DECIMAL),
        (20000::DECIMAL),
        (25000::DECIMAL)
) AS orders(amount);
```



Q21. Delivery Delay Function

Create:

`fn_delivery_delay_category(estimated_date, actual_date)`

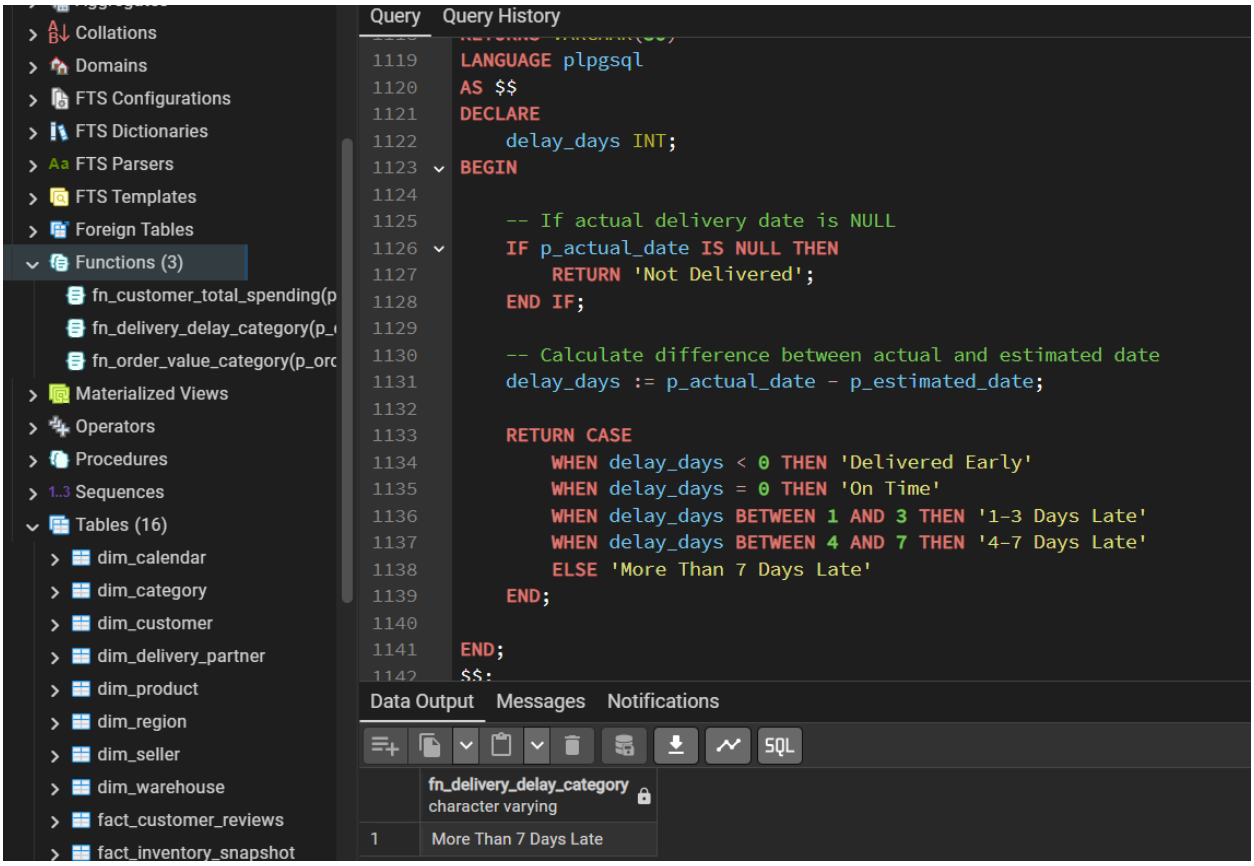
The function should return:

- Delivered Early
- On Time
- 1–3 Days Late
- 4–7 Days Late
- More Than 7 Days Late
- Not Delivered

Test the function with different date scenarios.

```sql
CREATE OR REPLACE FUNCTION fn_delivery_delay_category(
    p_estimated_date DATE,
    p_actual_date DATE
)
RETURNS VARCHAR(30)
LANGUAGE plpgsql
AS $$
DECLARE
    delay_days INT;
BEGIN

    IF p_actual_date IS NULL THEN
        RETURN 'Not Delivered';
    END IF;

    delay_days := p_actual_date - p_estimated_date;

    RETURN CASE
        WHEN delay_days < 0 THEN 'Delivered Early'
        WHEN delay_days = 0 THEN 'On Time'
        WHEN delay_days BETWEEN 1 AND 3 THEN '1–3 Days Late'
        WHEN delay_days BETWEEN 4 AND 7 THEN '4–7 Days Late'
        ELSE 'More Than 7 Days Late'
    END;

END;
$$;


SELECT fn_delivery_delay_category(
    '2026-08-10',
    '2026-08-08'
);

SELECT fn_delivery_delay_category(
```

```
    '2026-08-10',
    '2026-08-18'
);
```



🔄 Section 6 — Stored Procedures

Q22. Process Order Return Procedure

Create:

`sp_process_order_return()`

Parameters should include:

- Order Item ID

- Return Date
- Return Reason
- Quantity Returned
- Refund Amount

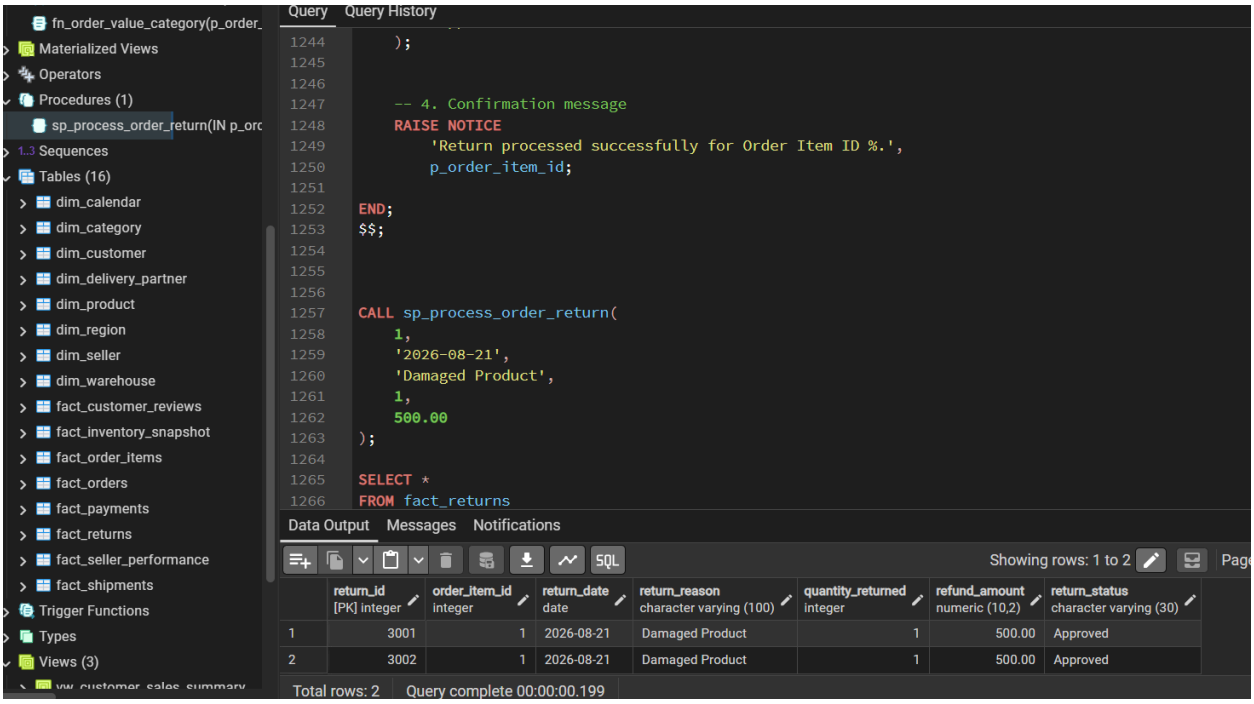
The procedure should:

1. Validate that the order item exists.
2. Validate that return quantity does not exceed ordered quantity.
3. Insert the return record.
4. Set an appropriate return status.

```
CREATE OR REPLACE PROCEDURE sp_process_order_return(
    p_order_item_id INT,
    p_return_date DATE,
    p_return_reason VARCHAR(100),
    p_quantity_returned INT,
    p_refund_amount DECIMAL(10,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_ordered_quantity INT;
BEGIN

    -- 1. Validate that order item exists
    SELECT quantity
    INTO v_ordered_quantity
    FROM fact_order_items
    WHERE order_item_id = p_order_item_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION
```

```
            'Order item ID % does not exist.',
            p_order_item_id;
    END IF;


    -- Validate return quantity
    IF p_quantity_returned <= 0 THEN
        RAISE EXCEPTION
            'Return quantity must be greater than 0.';
    END IF;


    IF p_quantity_returned > v_ordered_quantity THEN
        RAISE EXCEPTION
            'Return quantity (%) cannot exceed ordered quanti
ty (%).',
            p_quantity_returned,
            v_ordered_quantity;
    END IF;


    INSERT INTO fact_returns (
        order_item_id,
        return_date,
        return_reason,
        quantity_returned,
        refund_amount,
        return_status
    )
    VALUES (
        p_order_item_id,
        p_return_date,
        p_return_reason,
        p_quantity_returned,
        p_refund_amount,
        'Approved'
```

```
    );


    RAISE NOTICE
        'Return processed successfully for Order Item ID %.',
        p_order_item_id;

END;
$$;



CALL sp_process_order_return(
    1,
    '2026-08-21',
    'Damaged Product',
    1,
    500.00
);

SELECT *
FROM fact_returns
WHERE order_item_id = 1;
```



Q23. Process Customer Review Procedure

Create:

`sp_add_customer_review()`

Parameters:

- Order Item ID
- Customer ID
- Review Date
- Rating
- Review Comment

The procedure should:

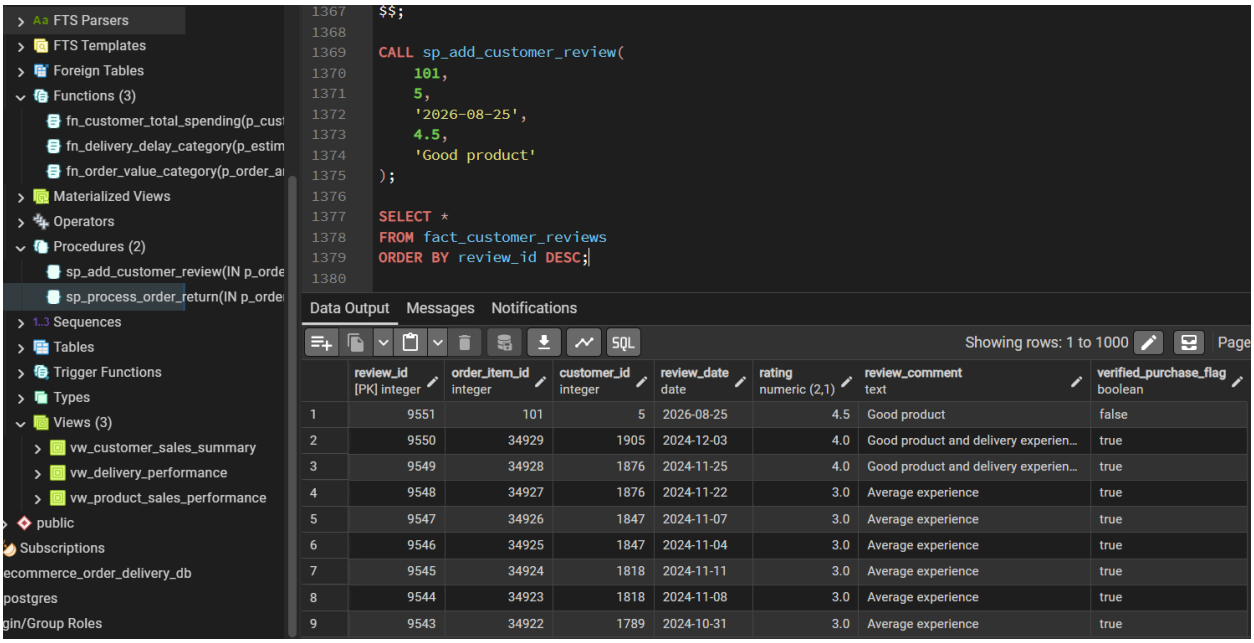
1. Validate that the order item exists.
2. Validate the rating is between **1 and 5**.
3. Insert the review.

4. Automatically determine `verified_purchase_flag` based on whether the customer actually purchased the item.

```
CREATE OR REPLACE PROCEDURE sp_add_customer_review(
    p_order_item_id INT,
    p_customer_id INT,
    p_review_date DATE,
    p_rating DECIMAL(2,1),
    p_review_comment TEXT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_order_exists BOOLEAN;
    v_customer_purchased BOOLEAN;
BEGIN

    SELECT EXISTS (
        SELECT 1
        FROM fact_order_items
        WHERE order_item_id = p_order_item_id
    )
    INTO v_order_exists;

    IF NOT v_order_exists THEN
        RAISE EXCEPTION
            'Order Item ID % does not exist.',
            p_order_item_id;
    END IF;


    IF p_rating < 1 OR p_rating > 5 THEN
        RAISE EXCEPTION
            'Rating must be between 1 and 5.';
```

```
    END IF;


    SELECT EXISTS (
        SELECT 1
        FROM fact_order_items foi
        JOIN fact_orders fo
            ON foi.order_id = fo.order_id
        WHERE foi.order_item_id = p_order_item_id
          AND fo.customer_id = p_customer_id
    )
    INTO v_customer_purchased;


    INSERT INTO fact_customer_reviews (
        order_item_id,
        customer_id,
        review_date,
        rating,
        review_comment,
        verified_purchase_flag
    )
    VALUES (
        p_order_item_id,
        p_customer_id,
        p_review_date,
        p_rating,
        p_review_comment,
        v_customer_purchased
    );


    RAISE NOTICE
        'Customer review added successfully for Order Item ID
%.',
        p_order_item_id;
```

```
END;
$$;

CALL sp_add_customer_review(
    101,
    5,
    '2026-08-25',
    4.5,
    'Good product'
);

SELECT *
FROM fact_customer_reviews
ORDER BY review_id DESC;
```



Q24. Shipment Status Processing Procedure

Create:

`sp_update_shipment_status()`

The procedure should update shipment status based on the available shipment dates.

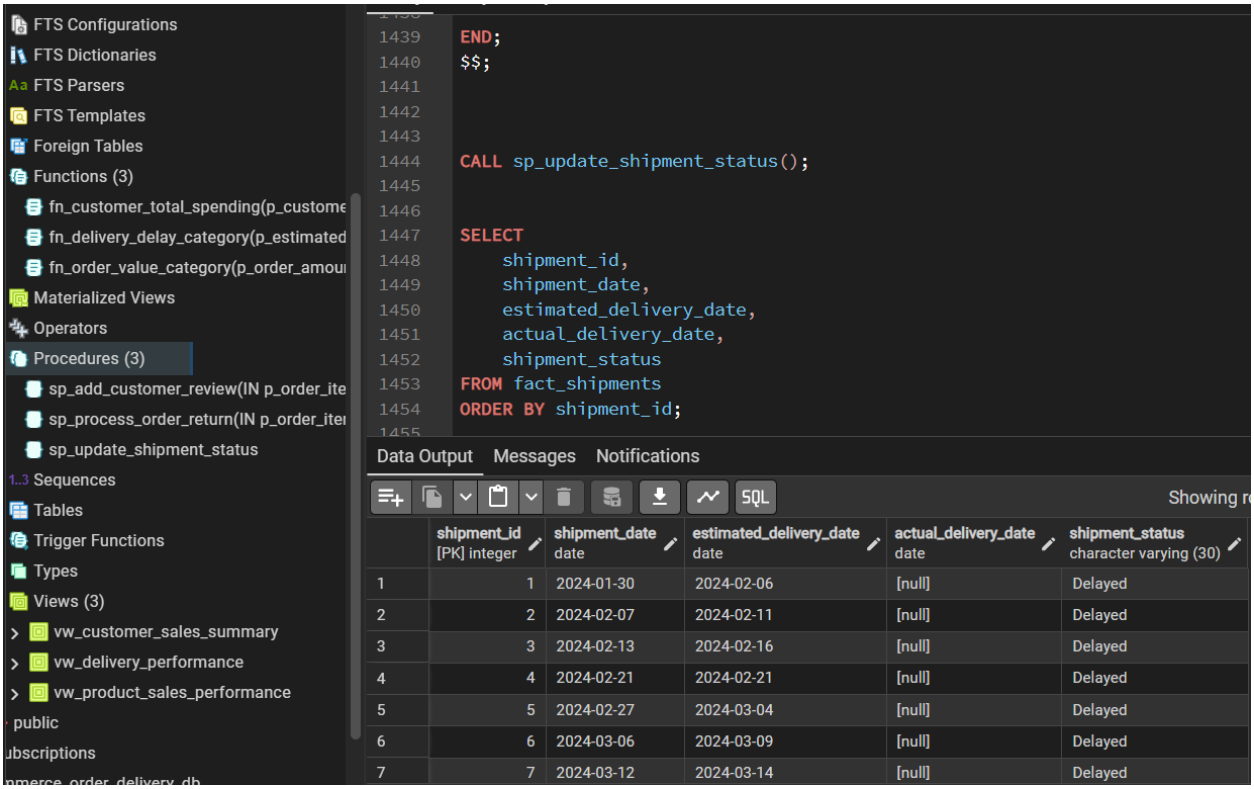
Suggested business rules:

```
Actual Delivery Date exists
        → Delivered

Shipment Date exists but Actual Delivery Date is NULL
        → In Transit / Shipped

No Actual Delivery + Expected Date passed
        → Delayed
```

Use appropriate `CASE` logic and date comparison.

```sql
CREATE OR REPLACE PROCEDURE sp_update_shipment_status()
LANGUAGE plpgsql
AS $$
BEGIN

    UPDATE fact_shipments
    SET shipment_status =
        CASE

            WHEN actual_delivery_date IS NOT NULL
                THEN 'Delivered'

            WHEN actual_delivery_date IS NULL
                 AND estimated_delivery_date < CURRENT_DATE
                THEN 'Delayed'

            WHEN shipment_date IS NOT NULL
                 AND actual_delivery_date IS NULL
```

```
                THEN 'In Transit'

            ELSE 'Shipped'

        END;

    RAISE NOTICE 'Shipment statuses updated successfully.';

END;
$$;



CALL sp_update_shipment_status();


SELECT
    shipment_id,
    shipment_date,
    estimated_delivery_date,
    actual_delivery_date,
    shipment_status
FROM fact_shipments
ORDER BY shipment_id;
```



🏆 Section 7 — Final Business Challenge

Q25. Executive E-Commerce Performance Report ⭐

Create one management-level report showing **performance of each product category**.

The final output should contain:

- Category
- Total Orders
- Units Sold
- Gross Sales
- Total Discount
- Net Sales

- Average Order Value
- Returned Units
- Return Rate
- Delivered Orders
- Delayed Orders
- On-Time Delivery %
- Average Customer Rating

Requirements

Your solution **must use**:

- ✅ At least **2 CTEs**
- ✅ Multiple JOINs
- ✅ `CASE`
- ✅ Date functions
- ✅ Aggregate functions
- ✅ Meaningful aliases

```
WITH sales_summary AS (

    SELECT
        c.category_id,
        c.category_name AS category,

        COUNT(DISTINCT foi.order_id) AS total_orders,

        SUM(foi.quantity) AS units_sold,

        SUM(foi.quantity * foi.unit_price) AS gross_sales,
```

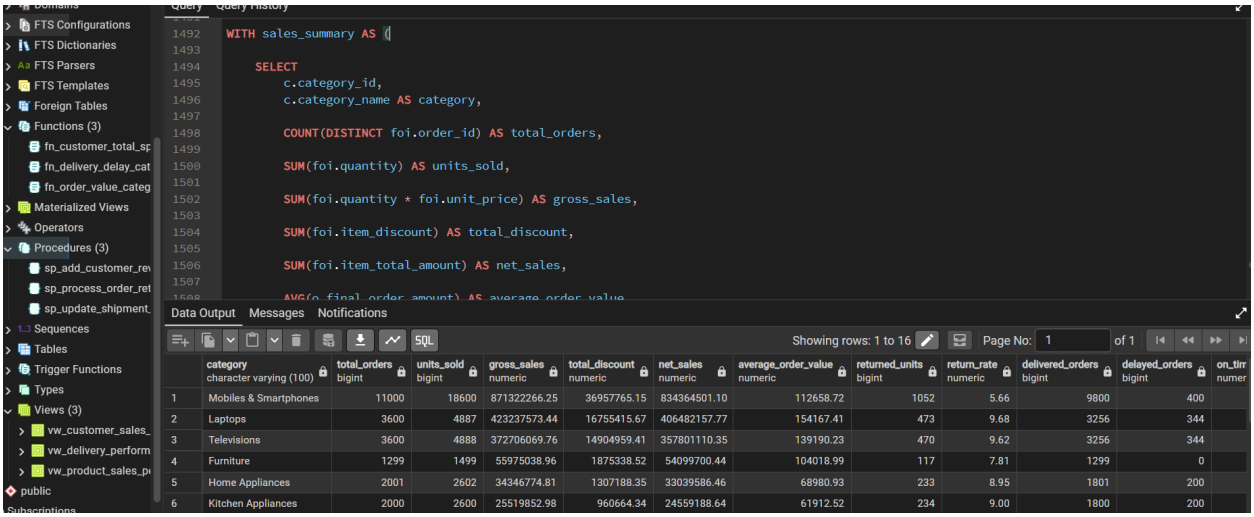
```
        SUM(foi.item_discount) AS total_discount,

        SUM(foi.item_total_amount) AS net_sales,

        AVG(o.final_order_amount) AS average_order_value

    FROM dim_category c

    JOIN dim_product p
        ON c.category_id = p.category_id

    JOIN fact_order_items foi
        ON p.product_id = foi.product_id

    JOIN fact_orders o
        ON foi.order_id = o.order_id

    GROUP BY
        c.category_id,
        c.category_name
),

return_summary AS (

    SELECT
        c.category_id,

        COALESCE(
            SUM(fr.quantity_returned),
            0
        ) AS returned_units

    FROM dim_category c

    JOIN dim_product p
        ON c.category_id = p.category_id
```

```
    JOIN fact_order_items foi
        ON p.product_id = foi.product_id

    LEFT JOIN fact_returns fr
        ON foi.order_item_id = fr.order_item_id

    GROUP BY
        c.category_id
),

delivery_summary AS (

    SELECT
        c.category_id,

        COUNT(
            DISTINCT CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                THEN fs.order_id
            END
        ) AS delivered_orders,

        COUNT(
            DISTINCT CASE
                WHEN fs.actual_delivery_date IS NULL
                     AND fs.estimated_delivery_date < CURRENT
_DATE
                THEN fs.order_id
            END
        ) AS delayed_orders,

        COUNT(
            DISTINCT CASE
                WHEN fs.actual_delivery_date IS NOT NULL
                     AND fs.actual_delivery_date
```

```
                        <= fs.estimated_delivery_date
                THEN fs.order_id
            END
        ) AS on_time_orders

    FROM dim_category c

    JOIN dim_product p
        ON c.category_id = p.category_id

    JOIN fact_order_items foi
        ON p.product_id = foi.product_id

    JOIN fact_shipments fs
        ON foi.order_id = fs.order_id

    GROUP BY
        c.category_id
),

rating_summary AS (

    SELECT
        c.category_id,

        AVG(fcr.rating) AS average_customer_rating

    FROM dim_category c

    JOIN dim_product p
        ON c.category_id = p.category_id

    JOIN fact_order_items foi
        ON p.product_id = foi.product_id

    JOIN fact_customer_reviews fcr
```

```sql
        ON foi.order_item_id = fcr.order_item_id

    GROUP BY
        c.category_id
)

SELECT
    ss.category,

    ss.total_orders,

    ss.units_sold,

    ROUND(ss.gross_sales, 2) AS gross_sales,

    ROUND(ss.total_discount, 2) AS total_discount,

    ROUND(ss.net_sales, 2) AS net_sales,

    ROUND(ss.average_order_value, 2) AS average_order_value,

    rs.returned_units,

    ROUND(
        rs.returned_units * 100.0
        / NULLIF(ss.units_sold, 0),
        2
    ) AS return_rate,

    COALESCE(ds.delivered_orders, 0) AS delivered_orders,

    COALESCE(ds.delayed_orders, 0) AS delayed_orders,

    ROUND(
        COALESCE(ds.on_time_orders, 0) * 100.0
        / NULLIF(
```

```
            COALESCE(ds.delivered_orders, 0),
            0
        ),
        2
    ) AS on_time_delivery_percentage,

    ROUND(
        COALESCE(rts.average_customer_rating, 0),
        2
    ) AS average_customer_rating

FROM sales_summary ss

LEFT JOIN return_summary rs
    ON ss.category_id = rs.category_id

LEFT JOIN delivery_summary ds
    ON ss.category_id = ds.category_id

LEFT JOIN rating_summary rts
    ON ss.category_id = rts.category_id

ORDER BY
    ss.net_sales DESC;
```



	category character varying (100)	total_orders bigint	units_sold bigint	gross_sales numeric	total_discount numeric	net_sales numeric	average_order_value numeric	returned_units bigint	return_rate numeric	delivered_orders bigint	delayed_orders bigint	on_tim numer
1	Mobiles & Smartphones	11000	18600	871322266.25	36957765.15	834364501.10	112658.72	1052	5.66	9800	400	
2	Laptops	3600	4887	423237573.44	16755415.67	406482157.77	154167.41	473	9.68	3256	344	
3	Televisions	3600	4888	372706069.76	14904959.41	357801110.35	139190.23	470	9.62	3256	344	
4	Furniture	1299	1499	55975038.96	1875338.52	54099700.44	104018.99	117	7.81	1299	0	
5	Home Appliances	2001	2602	34346774.81	1307188.35	33039586.46	68980.93	233	8.95	1801	200	
6	Kitchen Appliances	2000	2600	25519852.98	960664.34	24559188.64	61912.52	234	9.00	1800	200	